
VASP Workflow

User Manual

Browser GUI · CLI Agents · SLURM HPC

Keivan Esfarjani

v2.1 · September 2026 | <https://github.com/KeivanS/VASP-Flow>

Contents

1	Introduction	2
1.1	Key features	2
1.2	Requirements	2
2	Installation	3
2.1	default_paths — platform configuration	3
3	Browser GUI	3
3.1	Starting the GUI	3
3.2	Three-tab workflow	4
3.3	Setup tab — field guide	4
3.3.1	Structure and POTCAR	4
3.3.2	Functional	4
3.3.3	Spin / SOC	4
3.3.4	Other flags	5
3.4	Task checkboxes	5
3.5	Band structure options	5
3.6	DOS options	5
3.7	Workflow tab	5
3.8	Results tab	6
4	CLI Agents	6
4.1	Remote / headless setup (no GUI access)	6
4.2	vasp-agent.py — workstation	6
4.3	vasp-agent-slurm.py — SLURM clusters	7
4.4	Generated directory structure	7
4.5	Execution order	7
5	High-Throughput Screening (Materials Project)	8
5.1	Semimetal screen: ht-semimetals.py	10
6	instructions.txt Format	12
6.1	Example	12

6.2	Keyword reference	13
7	Methods	14
7.1	Functionals	14
7.2	Spin-orbit coupling	14
7.3	Magnetic materials (collinear spin)	14
7.4	GGA+U	15
7.5	Automatic k-mesh generation	15
8	Advanced INCAR Control	15
8.1	Raw INCAR passthrough	16
8.2	Explicit KPOINTS blocks	16
8.3	Relaxation → SCF hand-off	17
8.4	Command-line INCAR overrides (<code>-incar</code>)	17
8.5	Constant-pressure relaxation	17
9	K-Path for Band Structure	17
9.1	Auto-detection (recommended)	17
9.2	Manual override	18
10	DOS Projections	18
11	ELF Bond Profiles	19
11.1	Two-step ELF (ELF: <code>separate</code>)	19
12	Convergence Tests	20
12.1	Shifted test structure	20
12.2	Automatic parameter selection	20
13	Parallelization	21
13.1	Auto-defaults	21
13.2	Manual overrides	21
14	SLURM Cluster Settings	21
14.1	profiles/slurm.json	21
14.2	Per-project overrides (in <code>instructions.txt</code>)	22
15	Wannier90	22
16	Born Charges, Dielectric & Piezoelectric Tensors (DFPT)	22
16.1	Example	23
17	Phonons (phonopy)	23
18	LOBSTER Bonding Analysis (COHP / COBI / COOP)	23
18.1	LOBSTER projected DOS vs. the VASP DOS	24
19	Worked Examples	24
19.1	MoS ₂ monolayer with SOC	24
19.2	Nickel: magnetism + GGA+U	25
19.3	GaAs: phonons with NAC	25
19.4	High-pressure relaxation with custom INCAR tags	25
19.5	Bi ₂ Se ₃ : topological insulator	26

20 Common OUTCAR Checks	26
21 Troubleshooting	27
22 Version History	27
Addendum: High-Throughput Quick Reference	29

1 Introduction

VASP Workflow is a framework for setting up, running, and analysing DFT calculations with VASP. Two interfaces are provided:

- **Browser GUI** (`vasp-gui.py`) — a Flask web application: configure calculations through a form, run steps from the browser, view band-structure and DOS plots in real time.
- **CLI agents** (`vasp-agent.py` for workstations, `vasp-agent-slurm.py` for SLURM clusters) — read POSCAR and a plain-text `instructions.txt` file and generate all INCAR, KPOINTS, POTCAR, and job scripts automatically.

Both interfaces share the same `modules/` back-end.

1.1 Key features

- Functionals: PBEsol, PBE, R2SCAN, HSE06, RVV10, LDA, AM05
- Spin-orbit coupling (SOC) and collinear spin (ISPIN=2)
- Magnetic materials: correct MAGMOM per atom; spin-resolved band and DOS plots (spin-up / spin-down)
- GGA+U (Dudarev, LDAUTYPE=2) per element and orbital
- Automatic k-path detection via spglib (Setyawan–Curtarolo standard paths), with a geometric fallback
- Convergence testing: ENCUT and k-mesh sweeps
- DOS with orbital projections via `sumo`
- Phonons via phonopy, Wannier90 interface, DFPT Born charges
- LOBSTER bonding analysis (COHP/COBI/COOP) via the `08_lobster` step
- Smart KPAR/NCORE auto-defaults scaled with MPI task count
- SLURM workflow with dependency-chained job submission

1.2 Requirements

Package	Notes
VASP 5.4+	<code>vasp_std</code> , <code>vasp_nc1</code> (SOC), optionally <code>vasp_gam</code>
POTCAR library	PAW pseudopotentials in per-element folders
Python 3.8+	Conda environment recommended
Flask	<code>pip install flask</code> — GUI only
<code>sumo</code> ≥ 2.0	<code>pip install sumo</code> — band and DOS plots
MPI	<code>mpirun</code> , <code>mpiexec</code> , or <code>srun</code>
phonopy	optional; <code>conda install -c conda-forge phonopy</code>
wannier90.x	optional; <code>conda install -c conda-forge wannier90</code>

2 Installation

```
# 1. Clone the repository
git clone https://github.com/KeivanS/VASP-Flow.git
cd VASP-Flow

# 2. Install Python dependencies
pip install flask sumo

# 3. Optional: phonons and Wannier support
conda install -c conda-forge phonopy wannier90

# 4. First-time configuration (creates default_paths from template)
make setup
nano default_paths      # set VASP paths, POTCAR directory, MPI
                        command
```

2.1 default_paths — platform configuration

The file `default_paths` (Makefile key=value syntax) is read at startup and its variables are exported as environment variables that the GUI uses as initial defaults. Settings saved via the GUI *System Setup* page take precedence; `default_paths` acts as the fallback for new installs or CLI-only use.

```
PYTHON          = python3
VASP_STD        = /path/to/vasp_std
VASP_NCL        = /path/to/vasp_ncl
VASP_GAM        = /path/to/vasp_gam      # optional
VASP_POTCAR_DIR = /path/to/potpaw_PBE
MPI_LAUNCH      = mpirun -np            # or: srun
MPI_NP          = 16
WANNIER90_X     = wannier90.x
LOBSTER_X       = lobster               # lobster binary or full
path
VISUALIZER_X    = ~/bin/jmol             # Jmol, VESTA, OVITO, ...
```

Three-layer configuration precedence (highest wins):

1. GUI System Setup page → saved to `settings.json`; also auto-synced to `profiles/slurm.json` for SLURM agents.
2. `default_paths` → Makefile bootstrap, read on every `make run`.
3. Built-in code defaults.

`profiles/workstation.json` is used only when running `vasp-agent.py --profile workstation` directly from the command line without the GUI.

3 Browser GUI

3.1 Starting the GUI

```
make run      # launches on http://localhost:5001
```

On a SLURM cluster — run on the login node, open on your laptop via SSH port-forwarding:

```
# On your laptop:
ssh -L 5001:localhost:5001 username@cluster

# On the cluster login node:
make run

# Open http://localhost:5001 in your laptop's browser
```

3.2 Three-tab workflow

1. **Setup** — configure the project, then click *Generate Workflow*.
2. **Workflow** — run each calculation step; live VASP output streams to the browser.
3. **Results** — band-structure and DOS plots; OUTCAR analysis.

To return to a previous project use the *Select project* dropdown. → **Workflow** jumps to the run tab; **Edit & Regenerate** re-opens Setup with all settings pre-filled.

3.3 Setup tab — field guide

3.3.1 Structure and POTCAR

Field	Description
Project name	Folder created in the working directory.
POSCAR	Paste the crystal structure, or click <i>Load file</i> . Click <i>Visualize</i> to convert the current POSCAR to XYZ and open it in the structure viewer configured under System Setup (Jmol, VESTA, OVITO, ...).
POTCAR directory	Path to your PAW library. If an element has multiple variants (Ga vs Ga_d) a chooser appears.

3.3.2 Functional

Choice	Description
PBESol	Optimised for solids; recommended starting point.
PBE	Standard GGA; widely used.
R2SCAN	Meta-GGA; good accuracy/cost balance.
HSE06	Hybrid — accurate band gaps, significantly slower.
RVV10	van der Waals; for layered or molecular materials.
LDA / AM05	Rarely needed.

3.3.3 Spin / SOC

Choice	Effect
None	Non-magnetic.
Collinear spin (ISPIN=2)	Sets ISPIN = 2; reveals the <i>Magnetic moment / atom</i> field (Section 7.3).
SOC z/x/y	Sets LSORBIT, LNONCOLLINEAR, SAXIS; uses the vasp_nc1 binary.

3.3.4 Other flags

Field	Effect
Hexagonal BZ	Correct M point for hexagonal structures.
2D / slab	Sets $k_z = 1$ (Gamma-only out-of-plane).
GGA+U	Adds Hubbard U rows per element/orbital (Section 7.4).
Execution profile	Select <i>SLURM HPC</i> to generate <code>sbatch</code> scripts; edit <code>profiles/slurm.json</code> first.
MPI tasks	CPU cores per calculation.

3.4 Task checkboxes

Task	Directory	Notes
Structure relaxation	01_relax	IBRION=2; run first if geometry is not pre-optimised.
SCF	02_scf	Required before bands or DOS.
Band structure	03_bands	Line-mode KPOINTS; Section 9.
DOS	04_dos	Dense k-mesh; orbital projections via sumo.
DFPT	06_dfpt	Born charges, dielectric + piezoelectric tensors; Section 16.
Phonons	07_phonons	Phonopy finite-displacement; DFPT needed for NAC.
Wannier90	05_wannier	NSCF interface; Section 15.
LOBSTER	08_lobster	COHP/COBI/COOP bonding analysis; needs SCF; Section 18.

Note. Normal execution order: **relaxation** → **SCF** → **bands** → **DOS**. Each step copies POSCAR/CHGCAR from the previous one automatically.

3.5 Band structure options

Leave the *k-path* field blank to trigger **automatic detection** from the POSCAR lattice vectors (Section 9). Enter a path manually to override, e.g. **G-M-K-G**.

3.6 DOS options

Add projection rows to request orbital-resolved DOS:

Element: Mo Orbitals: **d** Element: S Orbitals: **p**

For spin-polarised calculations sumo plots spin-up positive and spin-down mirrored below zero automatically.

3.7 Workflow tab

Each step shows three controls: **Run** (submit the step), **Edit files** (open INCAR/KPOINTS/-POSCAR in the browser; a `.bak` is created on first edit), and a **status badge**. **Run All Steps** chains every step and stops on failure.

3.8 Results tab

Panel	Contents
Band structure	Spin-polarised: blue = $\uparrow$, red dashed = $\downarrow$, same k-axis, $E_F = 0$. Non-magnetic: single-colour sumo plot. If sumo fails (e.g. a pymatgen gap-detection crash) the GUI falls back to a built-in <code>vasprun.xml</code> plotter so a band plot still renders.
Cumulative DOS	Stacked filled-area by element. Spin-polarised: up positive, down mirrored.
Projected DOS	Orbital-resolved via sumo; spin-down negated automatically.
OUTCAR analysis	Energy, Fermi level, magnetic moment, forces.

4 CLI Agents

4.1 Remote / headless setup (no GUI access)

When running on a remote supercomputer where the browser GUI is not available, executable paths and MPI settings must be configured directly in `profiles/slurm.json` (for the SLURM agent) or `profiles/workstation.json` (for the workstation agent).

```
# On the cluster, after cloning VASP-Flow edit the file:
nano profiles/slurm.json
# or
vi profiles/slurm.json
```

The relevant keys (to be edited) are:

```
{
  "vasp_std":    "/path/to/vasp_std",
  "vasp_ncl":    "/path/to/vasp_ncl",
  "vasp_gam":    "/path/to/vasp_gam",
  "wannier90_x": "wannier90.x",
  "lobster_x":   "/path/to/lobster",
  "mpi_cmd":     "srun",
  "mpi_np":      128,
  "modules":     ["intel/2023", "vasp/6.4"],
  "slurm": {
    "partition": "standard",
    "nodes":     2,
    "ntasks_per_node": 64,
    "time":      "12:00:00",
    "account":   "your-project"
  }
}
```

The `modules` list is prepended as `module load ...` lines in every generated `run.sh`. Alternatively, all path keys can be set as environment variables in your `.bashrc` (`VASP_STD`, `LOBSTER_X`, etc.) and the profile values left as empty strings to inherit them.

Note: `profiles/slurm.json` is auto-regenerated whenever you click *Save Settings* in the GUI, so the authoritative source for GUI users is the System Setup page. On a headless cluster, edit the JSON file directly and leave it in place.

4.2 vasp-agent.py — workstation

The vasp agents read by default the POSCAR file and the instructions.txt file which contains the project name and type of calculation the user wants. It creates a folder by than name, from

which the calculations must be launched. Plots can be generated by running `analyze.sh` script from that folder.

```
./vasp-agent.py
./vasp-agent.py -i my_inst.txt -s struct.vasp

cd ProjectName
./run_all.sh           # runs every step in sequence
./analyze.sh          # plots after all steps complete
```

4.3 vasp-agent-slurm.py — SLURM clusters

```
./vasp-agent-slurm.py           # uses profiles/slurm.json
./vasp-agent-slurm.py --profile slurm

cd ProjectName
./submit_all.sh                 # sbatch with dependency chaining
squeue -u $USER                 # monitor jobs
./analyze.sh                    # run locally after jobs finish
```

Two-phase workflow when convergence tests are requested:

```
./submit_convergence.sh         # STEP 1 -- submit convergence jobs
# review results to decide on what to set for ENCUT and KPOINTS,
# then:
./submit_calculations.sh        # STEP 2 -- prompts ENCUT/k-mesh, submits
```

4.4 Generated directory structure

```
ProjectName/
  instructions.txt
  POSCAR
  POTCAR                      built once from $VASP_POTCAR_DIR
  01_relax/                   INCAR  KPOINTS  POSCAR  POTCAR  run.sh
  02_scf/
  03_bands/
  04_dos/
  08_lobster/                 if LOBSTER task selected (ISYM=0 NSCF +
    lobster)
  analysis/                   plots generated by analyze.sh
  run_all.sh                  workstation sequential runner
  submit_all.sh               SLURM job submission (dependency chained)
  analyze.sh                  post-processing: plots + OUTCAR extraction
```

4.5 Execution order

Mode	Behaviour
local	<code>run_all.sh</code> blocks — materials run strictly one-at-a-time.
slurm --chain (default)	Each material's SCF is submitted with <code>--dependency=afterok</code> on the previous one, so the cluster runs them sequentially even though all are queued up front.
slurm --no-chain	Materials are submitted independently and run whenever the scheduler allows.

Note. The driver fetches structures over the network (Materials Project), so run it on a node with internet — typically the login node. `runall.sh` performs no downloads and can run on a compute node.

5 High-Throughput Screening (Materials Project)

`ht-mp-scf.py` drives a batch of SCF + ELF calculations straight from Materials Project IDs. For every `mp-id` in the file `highthrouput_list` it downloads the **primitive** cell from MP, stages a per-material `instructions.txt` (an SCF task plus an INCAR `scf:` block setting `LELF = .TRUE.` for the electron localization function), and writes a `runall.sh` that calls the chosen agent to build the inputs and then runs or submits each job. Each SCF uses the automatic coarse density (1000 kpra, uniform mesh) by default; change it with `--kpra` (`fine`, or an integer such as 8000), per material via `KMESH_DENSITY` in `_ht_inputs/<id>/instructions.txt`, or give a fixed `--kmesh` grid.

ELF in two steps (default). The instructions carry `ELF: separate:` the SCF runs with the automatic `KPAR/NCORE` (full k-point parallelism), then a short `KPAR = 1` restart in `02_scf/elf/` reads the converged `WAVECAR/CHGCAR` and writes `ELFCAR`, copied back to `02_scf/ELFCAR` (Section 11.1). Previously the whole SCF ran with `KPAR = 1` because of `LELF`.

```

pip install pymatgen mp-api          # MP download + primitive
standardisation
export MP_API_KEY=...                # Materials Project API key
export VASP_POTCAR_DIR=...           # POTCAR library (used by the
agent)

# one mp-id per line (blank lines and # comments ignored)
printf 'mp-149\nmp-2534\n' > highthrouput_list
# or you can populate your highthrouput_list in advance

./ht-mp-scf.py --agent local --mpi 16      # workstation
./ht-mp-scf.py --agent slurm --profile slurm # cluster (chained by
default)

./runall.sh                             # build inputs + run/submit
every job

```

The `ELFCAR` for each material is written to `<mp-id>/02_scf/`. A summary `_ht_inputs/elements_Z.txt` tabulates every element in the screening set with its atomic number Z (from a built-in table — no MP query), plus a per-material breakdown.

Option	Meaning
<code>--agent</code>	<code>local</code> (vasp-agent.py) or <code>slurm</code> (vasp-agent-slurm.py); prompts if omitted.
<code>--list</code>	ID file; default <code>highthrouput_list</code> .
<code>--functional</code>	PBE (default), PBEsol, R2SCAN, ...
<code>--mpi</code>	MPI tasks per job (workstation; SLURM uses nodes $\times$ ntasks-per-node).
<code>--encut</code>	Optional ENCUT override (eV).
<code>--kmesh</code>	Fixed SCF Γ -mesh for every material, e.g. '8 8 8'; overrides <code>--kptra</code> . Not recommended for a set of different cell shapes (the mesh is then not uniform).
<code>--kptra</code>	k-point density (default <code>coarse</code> = 1000): <code>coarse</code> , <code>fine</code> (5000) or an integer (e.g. 8000).
<code>--kpar, --ncore</code>	Force KPAR and/or NCORE in the SCF (default: automatic; give one and the other is derived). The ELF pass always uses KPAR = 1.
<code>--nodes, --ntasks-per-node, --partition, --walltime, --account</code>	SLURM overrides written into each <code>instructions.txt</code> .
<code>--chain / --no-chain</code>	SLURM only; see below.

```
# example: SLURM, 1 node x 40 cores, density 8000, automatic
#          KPAR/NCORE
./ht-mp-scf.py --agent slurm --nodes 1 --ntasks-per-node 40 \
  --kptra 8000 --walltime 12:00:00 --account elmgrou
# same, but force NCORE=2 (KPAR is derived: 20)
./ht-mp-scf.py --agent slurm --nodes 1 --ntasks-per-node 40 \
  --kptra 8000 --ncore 2 --walltime 12:00:00 --account elmgrou
```

5.1 Semimetal screen: `ht-semimetals.py`

`ht-semimetals.py` builds a complete SLURM-ready `highthroughput_*` directory (SCF → bands → LOBSTER, one `job.sbatch` per material) from a list file such as `lists/semimetals_test.txt`. No DFPT step: with a zero gap, dielectric tensors and Born charges are ill-defined and the linear-response loop tends not to converge. No ELF either (`ELF: off`).

ht-mp-scf.py vs. ht-semimetals.py. `ht-mp-scf.py` is the general driver: SCF + ELF (+ optional LOBSTER) per material, run through `runall.sh` on a workstation or on SLURM (one job per step, chained with `afterok`), with one resource setting for all materials. `ht-semimetals.py` is a screening pipeline built on top of it: SCF + bands + LOBSTER in *one* SLURM job per material, a per-material ENCUT ($1.3 \times \text{max ENMAX}$) and job size, magnetic starting moments for 3d elements, a throttled `submit_all.sh` with resume, `status.sh/retune.sh/collect_results.sh`, and a POTCAR manifest so the directory can be generated on one machine and run on another. It reuses `ht-mp-scf.py` and `modules/`, so run it from a clone of the repository (`git pull` first) on a machine with internet access, `pymatgen`, `mp-api`, a Materials Project key and the POTCAR library:

```

pip install pymatgen mp-api spglib
export MP_API_KEY=...                # real key
    (materialsproject.org/api)
export VASP_POTCAR_DIR=/path/to/potpaw_PBE

python3 ./ht-semimetals.py --list lists/semimetals_test.txt \
    --out highthroughput_test --api-key "$MP_API_KEY" \
    --kpara 8000 --single-node --kpar 20 --ncore 2

cd highthroughput_test && ./submit_all.sh    # builds missing
    POTCARs, then throttled sbatch
./status.sh                                # progress
./analyze_all.sh                            # redo plots (each job
    runs analyze.sh itself)
./retune.sh --time 06:00:00                 # change walltime in
    place

```

Option	Meaning
<code>--list, --out</code>	Input list (one mp-id per line) and output directory.
<code>--kpra N</code>	k-points per reciprocal atom; overrides <code>--kspacing</code> .
<code>--single-node</code>	Every material on one node with <code>--cores-per-node</code> (default 40) ranks.
<code>--kpar, --ncore</code>	Force KPAR/NCORE in the SCF, bands and LOBSTER INCARs (default: automatic); DFPT al-

6 instructions.txt Format

6.1 Example

```
Project: My material name

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Fe-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Fe-d and O-p)

Convergence: Test k-points 6x6x6 to 12x12x12
              ENCUT 400-600 eV

MAGMOM: 4.0
NSW: 100
EDIFFG: -0.01
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 12:00:00
```

The parser is case-insensitive and searches the entire file, so section order is flexible.

6.2 Keyword reference

Keyword	Meaning
Project:	Project name; becomes the output folder.
MAGMOM:	Initial magnetic moments in μ_B : one number for all atoms (4.0, -3.0), per element (Fe=4.0, O=0.6) or a per-atom list (2*4.0 2*-4.0). Any MAGMOM line switches spin polarisation on; default 2.0 when only “collinear magnetization” is given (Section 7.3).
ISIF:	Relaxation type: 3=full, 2=ions only, 4=fixed volume.
NSW:	Max ionic steps. Default 100.
EDIFFG:	Force criterion in eV/Å. Default -0.01.
ENCUT:	Manual plane-wave cutoff in eV.
KMESH:	Explicit Gamma k-mesh overriding the automatic density, e.g. 8 8 8 or 12 (cubic). Applies to every automatic mesh (relax/SCF/DOS); for 2-D give the third value (12 12 1).
KMESH_DENSITY:	Automatic k-mesh density tier: coarse (1000 kpra, default for relaxation and convergence tests) or fine (5000 kpra, default for SCF/-DOS/spectroscopic steps). $kpra = N_k \times N_{atoms}$ where $N_k = N_x N_y N_z$; subdivisions scale as $N_i \propto \mathbf{b}_i^* $. An integer is taken as the kpra target itself (also KPRA:, KPOINTS_DENSITY:). Overridden by explicit KMESH and by KPOINTS blocks.
RELAX_KMESH_DENSITY:	Density of the relaxation mesh (coarse , fine or an integer); default coarse whatever the SCF uses.
PRESSURE:	External pressure for constant- P relaxation, e.g. 10 GPa or 50 kbar. Forces ISIF=3, IBRION=2, PSTRESS (Section 8.5).
INCAR: ... END_INCAR	Block of literal INCAR tags merged into the generated INCAR(s); optionally per-step (Section 8.1).
NKPTS:	k-points per band segment. Default 40.
MPI:	Total MPI tasks (nodes $\times$ cores/node).
KPAR:	Global k-point parallelisation.
NCORE:	Global cores-per-orbital.
SCF_KPAR: etc.	Per-step KPAR: also BANDS_, DOS_, RELAX_, PHONONS_ (DFPT is forced to KPAR = NCORE = 1).
SCF_NCORE: etc.	Per-step NCORE; same prefixes.
NODES:	Compute nodes (SLURM).
NTASKS_PER_NODE:	MPI tasks per node (SLURM).
PARTITION:	SLURM partition.
WALLTIME:	Time limit, e.g. 12:00:00.
ACCOUNT:	SLURM account / allocation.
DFPT_EDIFF:	EDIFF for DFPT. Default 1E-8.
PHONONS_DIM:	Supercell, e.g. 2 2 2.
PHONONS_MESH:	q-mesh, e.g. 20 20 20.
PHONONS_DISP:	Displacement in Å. Default 0.01.
PHONONS_BAND:	High-symmetry path for phonon bands.
PHONONS_NAC:	FALSE to disable NAC.
WANNIER_NUM_WANN:	Number of Wannier functions.
WANNIER_PROJ:	Projections, e.g. Ga:s;As:p.
WANNIER_EWIN:	Energy window, e.g. -5.0 to 15.0.

7 Methods

7.1 Functionals

```
Methods: PBEsol functional      # recommended for solids
Methods: R2SCAN functional      # meta-GGA
Methods: HSE06 functional       # hybrid -- more accurate but slower
Methods: RVV10 functional       # van der Waals
```

Default ENCUT values: PBE 400 eV, PBEsol 450 eV, R2SCAN 680 eV, HSE06 400 eV, RVV10 450 eV.

7.2 Spin-orbit coupling

```
Methods: SOC with magnetization in z-direction
```

Uses vasp_ncl. Sets LSORBIT, LNONCOLLINEAR, and SAXIS.

7.3 Magnetic materials (collinear spin)

```
Methods: collinear magnetization
MAGMOM: 4.0
```

Sets ISPIN = 2 and MAGMOM = N * value in every INCAR, where N is the atom count from the POSCAR. Three forms of the MAGMOM: line are understood (any of them switches spin polarisation on, with or without the words “collinear magnetization”):

```
MAGMOM: 4.0                # uniform; negative values keep their sign
MAGMOM: Fe=4.0, O=0.6      # per element (unlisted elements start at 0)
MAGMOM: 2*4.0 2*-4.0      # per atom in POSCAR order, n*value
shorthand
```

A per-atom list must have exactly N entries; the agent stops with an explanatory message rather than silently falling back to a default.

Tip. Typical starting moments: Fe, Co $\approx 4\text{--}5 \mu_B$; Ni $\approx 2 \mu_B$; Cr, Mn $\approx 3\text{--}4 \mu_B$; rare earths $\approx 7 \mu_B$. VASP converges the self-consistent value automatically.

Antiferromagnets — give the alternating signs directly, e.g. MAGMOM: 4.0 -4.0 4.0 -4.0.

With SOC (Methods: SOC with magnetization in x-direction) the moments are 3N components in the non-collinear MAGMOM; the same MAGMOM: values are rotated onto the requested direction (without this VASP would start every moment along its default (1,1,1)). A list of exactly 3N numbers is used as the raw x, y, z components.

Spin-resolved plots produced automatically after DOS/bands run:

- *Cumulative DOS* — spin-up above the x-axis; spin-down mirrored below zero; elements colour-coded; solid = $\uparrow$, dashed = $\downarrow$.
- *Band structure* — spin-up in blue, spin-down in red dashed, same k-axis, $E_F = 0$.
- *Projected DOS* — sumo natively negates the spin-down channel in the orbital-resolved PDF.

Check the converged moment:

```
grep "magnetization_(x)" OUTCAR | tail -5
```

7.4 GGA+U

```
Methods: GGA+U with U=4.0 on Fe-d orbitals
         GGA+U with U=3.0 on Ni-d orbitals
         GGA+U with U=6.0 on Ce-f orbitals
```

Dudarev scheme (LDAUTYPE=2); $U_{\text{eff}} = U - J$ with $J = 0$ set automatically. In the Dudarev formalism the parameter labelled U is already U_{eff} , so no separate J entry is needed. Combine freely with **collinear magnetization**.

The automatic lookup is triggered whenever the structure contains a transition-metal or rare-earth element *and* at least one electronegative anion from the set O, F, S, Se, Te, Cl, Br, I. This covers oxides, sulfides (e.g. AgCrP₂S₆), selenides, tellurides, and halides. U values are read from `hubbard_u_defaults.csv`; elements absent from the table (e.g. Ag with a filled d^{10} shell) receive no correction.

7.5 Automatic k-mesh generation

All Gamma-centred meshes are generated automatically from the POSCAR geometry. Two density tiers are available via the **KMESH_DENSITY** key (Section 6.2):

Tier	kpra	Typical use	Notes
coarse	1000	Relaxation, convergence tests	Agent k-point sweeps start here
fine	5000	SCF, DOS, spectroscopic steps	Default for all production steps

kpra (k-points per reciprocal atom) = $N_x N_y N_z \times N_{\text{atoms}}$. Given a target kpra and the reciprocal lattice vectors $\mathbf{b}_i^*$ (columns of $2\pi(\mathbf{A}^{-1})^\top$), the mesh follows from one formula, with no search:

$$N_i = \text{round}_2(s |\mathbf{b}_i^*|), \quad s = \left(\frac{N_k}{|\mathbf{b}_1^*| |\mathbf{b}_2^*| |\mathbf{b}_3^*|} \right)^{1/3}, \quad N_k = \text{kpra} / N_{\text{atoms}},$$

where round_2 rounds to the nearest *even* integer (minimum 2), so that BZ boundary points ($k = 1/2$) are on the mesh. Because every axis uses the same s , the spacing $|\mathbf{b}_i^*|/N_i$ is the same along all three axes up to that rounding (a uniform mesh), axes with equal $|\mathbf{b}_i^*|$ get equal N_i , and $N_1 N_2 N_3 N_{\text{atoms}} \approx \text{kpra}$. For hexagonal lattices ($a = b$, $\gamma = 120^\circ$, detected from the POSCAR) the two in-plane N are rounded to multiples of 6 (6, 12, 18, ...) so that K and M lie on the mesh. For 2-D slabs the same formula is used in the plane (square root instead of cube root) and $N_z = 1$, so the in-plane mesh carries the whole target. The first line of each generated KPOINTS reports the mesh and the kpra actually reached. Any integer may be given instead of **coarse/fine** (**KMESH_DENSITY**: 3000); the relaxation has its own density (**RELAX_KMESH_DENSITY**, default coarse).

DOS and LOBSTER meshes: always $2N_x \times 2N_y \times 2N_z$ (the SCF ratios are preserved; a 2-D slab keeps $N_z = 1$), ensuring denser sampling for accurate spectral features and smoother COHP/COBI/COOP curves. Because the LOBSTER NSCF runs with **ISYM**=0 there is no symmetry reduction, so doubling in 3-D multiplies its k-point count by ~ 8 ; edit `08_lobster/KPOINTS` if that is too costly. The DFPT mesh is forced to match the SCF mesh exactly.

An explicit **KMESH**: $N_1 \ N_2 \ N_3$ entry overrides the automatic density for all steps, and a **KPOINTS** block (Section 8.2) overrides everything for the step it names.

8 Advanced INCAR Control

Any VASP keyword can be specified explicitly — nothing is hard-wired. Three routes, in increasing precedence: (1) an **INCAR**: ... **END_INCAR** block in the instructions file (global or per-step); (2) the agents' **-incar** command-line option; (3) direct editing — in the GUI every

generated INCAR is editable per step (Workflow tab → *Edit files*, with an automatic `.bak` backup), and of course the files on disk can be edited before running. This section covers the first two plus **constant-pressure relaxation**.

8.1 Raw INCAR passthrough

Place literal INCAR lines — in ordinary VASP syntax — inside an `INCAR: ... END_INCAR` block in your `instructions.txt`. They are merged into the generated INCAR(s):

```
INCAR:                                # global block -- applies to every step
  LREAL = Auto
  ALGO   = Fast
  NELMIN = 6
END_INCAR
```

A tag that the agent already writes is **overwritten in place** (so `ALGO = Fast` above replaces the default `ALGO = Normal`); any tag the agent does not normally emit is appended under a clearly labelled `# User INCAR overrides` section, so nothing is silently lost.

To target a single step, name it on the header line. Recognised steps are `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`, `lobster`:

```
INCAR scf:                            # applies only to 02_scf
  EDIFF = 1E-8
  SIGMA = 0.10
END_INCAR
```

Note. Precedence: a *per-step* block beats the *global* block, and both beat the agent's generated defaults. This is the recommended way to set tags the parser has no dedicated keyword for (e.g. `LMAXMIX`, `ADDGRID`, `LVTOT`, `IVDW`, `AMIX`). Full-line `#` comments in the instructions file are ignored by the parser, so a task keyword inside a comment does not enable that task.

8.2 Explicit KPOINTS blocks

The same block syntax writes a complete KPOINTS file, *unchanged*, for a step:

```
KPOINTS scf:                          # or KPOINTS_SCF:
  my SCF mesh                          # first line = KPOINTS comment
  0
  Gamma
  9 9 5
  0 0 0
END_KPOINTS
```

Steps: `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`, `lobster`; an unnamed KPOINTS: block applies to every step except the band path. Explicit blocks (INCAR and KPOINTS alike) **override every other setting**: keywords, `KMESH`, the density tier and the automatic `KPAR/NCORE`. Steps derived from the SCF mesh — `DOS` and `LOBSTER` (2×), `DFPT` and `Wannier90` (1×) — follow an explicit SCF block. A marker file `.explicit_kpoints` is left beside such a KPOINTS so that the convergence phase-2 k-mesh patching never overwrites it. `KPAR` and `NCORE` written in an INCAR block also steer the companion value (e.g. `KPAR = 2` on 16 ranks gives `NCORE = 2`). In the GUI both block kinds are entered under *Setup* → *Advanced* — *explicit INCAR / KPOINTS*.

8.3 Relaxation → SCF hand-off

The relaxation starts from scratch (`ISTART = 0`, `ICHARG = 2`) and writes `WAVECAR` and `CHGCAR`. `02_scf/copy_from_relax.sh` copies the relaxed geometry and the `CHGCAR`; if the SCF k-mesh, spin and cutoff equal the relaxation's it also copies the `WAVECAR` and the SCF starts from both (`ISTART = 1`, `ICHARG = 1`), otherwise it starts from the `CHGCAR` alone (`ISTART = 0`, `ICHARG = 1`) because wavefunctions on different k-meshes are not interchangeable. The decision is made at run time, so k-meshes edited or patched after generation are handled. Putting `ISTART` or `ICHARG` in an `INCAR scf:` block disables the mechanism.

8.4 Command-line INCAR overrides (-incar)

Both `vasp-agent.py` and `vasp-agent-slurm.py` accept a repeatable `-incar` option that injects explicit VASP tags at invocation time, without touching the instructions file:

```
./vasp-agent.py -i instructions.txt -s POSCAR \
  --incar "ALGO=Fast;NELMIN=6" \           # every step
  --incar "scf:NBANDS=64"                  # 02_scf only
```

Multiple tags in one option are separated by `;` (not commas — values such as `MAGMOM` may contain commas). An optional step prefix (`relax/scf/bands/dos/wannier/dfpt/phonons/lobster`) targets a single step. Command-line tags are applied *after* the instructions-file blocks, so they win on conflicts.

Note. Exceptions: physics-critical guards are re-imposed regardless of overrides — `ISYM=2` and `KPAR=NCORE=1` in `06_dfpt`, and the symmetry-off `ISYM` in `08_lobster`.

8.5 Constant-pressure relaxation

A single `PRESSURE` line turns the relaxation into a constant-pressure optimisation: the agent forces `ISIF = 3` (cell shape + volume + ions) and `IBRION = 2`, and imposes the target pressure through `PSTRESS`.

```
PRESSURE: 10 GPa          # also accepted: "5 kbar", "constant
  pressure 8"
```

VASP's `PSTRESS` tag is in **kBar** (1 GPa = 10 kBar). The agent converts automatically; if no unit is given, **GPa is assumed**. The resulting `01_relax/INCAR` reads, e.g.:

```
# Constant-pressure relaxation @ 10 GPa (PSTRESS = 100 kBar)
IBRION = 2
ISIF    = 3
NSW     = 100
EDIFFG  = -0.01
PSTRESS = 100 ! external pressure in kBar (10 GPa)
```

Tip. `PSTRESS` adds the PV term so the relaxed cell sits at the requested external pressure. Use a tight `EDIFFG` and a well-converged `ENCUT`; the Pulay stress error grows at high pressure, so verify with a higher `ENCUT` before trusting the equilibrium volume.

9 K-Path for Band Structure

9.1 Auto-detection (recommended)

Omit the path in `instructions.txt` or leave the field blank in the GUI:

```
Tasks: band structure      # no "along ..." -> auto-detect from
POSCAR
```

Since v1.6 the generator first determines the *actual space group* of the POSCAR with spglib (via pymatgen's `KPathSetyawanCurtarolo`) and uses the standard Setyawan–Curtarolo high-symmetry path for that space group. Every high-symmetry point is transformed through Cartesian reciprocal space into the reciprocal basis of *this* POSCAR, so the path is correct for the actual cell used in the SCF regardless of orientation or setting.

If spglib or pymatgen is not installed (or the symmetry analysis fails), the generator falls back to the geometric classifier below: it reads the POSCAR lattice vectors, computes cell lengths and angles, and selects a default path by Bravais type. 2-D structures always use the geometric route so the in-plane path is preserved.

Type	Detection criterion	Default path
cF (FCC)	$a=b=c$, angles $\approx 60^\circ$	G-X-W-K-G-L-U-W
cI (BCC)	$a=b=c$, angles $\approx 109.5^\circ$	G-H-N-G-P-H
cP (cubic)	$a=b=c$, 90°	G-X-M-G-R-X
hP (hexagonal)	$a \approx b$, $\gamma = 120^\circ$	G-M-K-G-A-L-H-A
tP (tetragonal)	$a \approx b \neq c$, 90°	G-X-M-G-Z-R-A-Z
oP (orthorhombic)	90° , $a \neq b \neq c$	G-X-S-Y-G-Z-U-R-T-Z
rP (rhombohedral)	$a=b=c$, equal angles	G-T-F-G-L
mP (monoclinic)	$\alpha = \gamma = 90^\circ$, $\beta \neq 90^\circ$	G-Y-A-Z-G-B-D

Note. For **conventional** FCC or BCC cells (right angles, 4 or 2 atoms), auto-detection returns the cubic (cP) path. Use the **primitive** cell for an unfolded FCC/BCC band structure — the primitive FCC cell has angles $\approx 60^\circ$ and is recognised automatically.

The detection route appears in the KPOINTS comment:

```
Line-mode KPOINTS [spglib ... -- auto-detected] # spglib route
Line-mode KPOINTS [FCC -- auto-detected]       # geometric
fallback
```

9.2 Manual override

```
Tasks: band structure along G-X-W-K-G-L
```

Label	Coords	Use	
G	(0, 0, 0)	all	Γ
M	(0.5, 0.5, 0)	cubic, tet	(0.5, 0, 0) in hexagonal
K	(1/3, 1/3, 0)	hexagonal	
R	(0.5, 0.5, 0.5)	cubic	
Z	(0, 0, 0.5)	general	
A	(0, 0, 0.5)	hex, tet	
L	(0.5, 0, 0.5)	hexagonal	
H	(1/3, 1/3, 0.5)	hexagonal	
S	(0.5, 0.5, 0)	orthorhombic	
W	(0.5, 0.25, 0.75)	FCC primitive	
P	(0.25, 0.25, 0.25)	BCC primitive	
N	(0, 0, 0.5)	BCC primitive	

10 DOS Projections

All of the following forms are accepted:

```
DOS (projected on Mo-d and S-p)
DOS with projections on Ni:d and Ni:s
DOS with projection on Fe-d and O-p
```

Orbital channels: s, p, d, f. Separator is either - or :. Sumo generates an orbital-resolved PDF in `analysis/`.

11 ELF Bond Profiles

If an SCF calculation writes an `ELFCAR` (electron localization function, requested with `LELF = .TRUE.` — the high-throughput driver sets this automatically), the utility `elf_bonds.py` plots ELF along each non-equivalent nearest-neighbour bond, sampled from one atom centre to the other:

```
./elf_bonds.py [ELFCAR] [--out PREFIX] [--nn-scale 1.2] [--npoints
200]
```

It parses the `ELFCAR` grid directly (only numpy + matplotlib), groups bonds by element pair and length (rounded to 0.01 Å) to pick the non-equivalent ones, and interpolates ELF along each bond with periodic boundary conditions. Output is `FORMULA_elf_bonds.png/pdf` (the prefix defaults to the reduced formula, e.g. `MoSe2_elf_bonds.pdf`; override with `-out`) showing ELF vs. distance along the bond, with the `ELF = 0.5` homogeneous-electron-gas reference marked. `analyze.sh` runs it automatically when `02_scf/ELFCAR` is present, and the GUI Results tab shows the same plot whenever the SCF wrote an `ELFCAR`.

Tip. Reading the profile: ELF is high (near 1) where electrons are paired and localized. A covalent bond therefore shows an ELF *maximum* in the bonding region *between* the atoms (e.g. Si-Si ≈ 0.95 at mid-bond), not at the nuclei. Because VASP builds the `ELFCAR` from the pseudo (valence) density, the value *at* an atom centre is a PAW artifact — often low, even ≈ 0 for heavy atoms — so read the bonding region, not the endpoints.

Note. VASP does *not* compute ELF for non-collinear / spin-orbit runs (LSORBIT/LNONCOLLINEAR); it prints "WARNING: ELF not implemented for non collinear case" and writes an all-zero `ELFCAR`. `elf_bonds.py` detects this and reports it rather than producing a blank plot — re-run the SCF *without* SOC to get ELF.

11.1 Two-step ELF (ELF: separate)

`LELF` only works with `KPAR = 1`, which removes the k-point parallelism that makes an SCF on a small cell fast. With

```
ELF: separate          # or  ELF: two-step
```

the ELF is produced in two steps inside `02_scf`, in the same job:

1. The SCF runs normally — automatic `KPAR/NCORE`, no `LELF`, `LWAVE/LCHARG=.TRUE.`..
2. `run_elf.sh` (called by `run.sh` after VASP) copies the converged `WAVECAR`, `CHGCAR`, `POSCAR`, `KPOINTS` and `POTCAR` into `02_scf/elf/` and runs VASP there with `02_scf/elf/INCAR`: the SCF `INCAR` (same `ENCUT`, smearing, `MAGMOM`, `+U`, user overrides) plus `ISTART=1`, `ICHARG=1`, `LELF=.TRUE.`, `LWAVE/LCHARG=.FALSE.`, `KPAR = 1` and `NCORE` from the rule for one k-group. `NBANDS` is set at run time to the value the SCF used (read from its `OUTCAR`), so the `WAVECAR` is read as is. Starting from converged wavefunctions, this restart needs only a few electronic steps.

3. ELFCAR is copied back to 02_scf/ELFCAR, where `analyze.sh`, `elf_bonds.py` and the GUI look for it; the copied WAVECAR/CHGCAR in `elf/` are deleted.

If the SCF left no WAVECAR/CHGCAR, the ELF pass is skipped with a message. The DOS, bands and LOBSTER steps still read 02_scf/CHGCAR, which the ELF pass does not touch. `ht-mp-scf.py` uses this mode by default; write `ELF: on` in `instructions.txt` for the old single run.

12 Convergence Tests

```
# Range (auto-steps)
Convergence: Test k-points from 6x6x1 to 18x18x1      # 2D material
               Test k-points from 6x6x6 to 12x12x12    # 3D material
               ENCUT 400-600 eV

# Explicit mesh list
Convergence: Test k-points 6x6x3, 12x12x6, 18x18x9

# ENCUT: all of these are equivalent (step defaults to 50 eV)
Test ENCUT 350-650      Test ENCUT 350 650      Test ENCUT 350 to 650
Test ENCUT 350,650      Test ENCUT 350:50:650    # start:step:stop
```

The `start:step:stop` form is the only one that sets the step; the others sweep in 50 eV increments. A convergence line that cannot be parsed now emits a `[warn]` on stderr rather than silently generating no tests at all.

Convergence INCARs are trimmed to what the test actually reads — the total energy in OUTCAR. LWAVE, LCHARG, LORBIT and LELF are dropped, which avoids writing a full WAVECAR per point and, because LELF pins KPAR to 1, restores k-point parallelism for the sweep.

Results in 00_convergence/kpoints/ and 00_convergence/encut/.

```
./run_convergence.sh      # workstation
./submit_convergence.sh   # SLURM
./analyze_convergence.sh  # plots: energy / pressure / forces
```

12.1 Shifted test structure

The POSCAR used in 00_convergence has **atom 1 displaced by (0.01, 0.02, 0.03) Å (Cartesian)** so that the force on it is non-zero and its convergence can be tracked — at the equilibrium (high-symmetry) positions the forces would vanish identically and the force panel would be meaningless. The displacement is converted through the lattice for Direct-mode POSCARs. All production steps (01_relax, 02_scf, ...) use the *original, unshifted* positions.

12.2 Automatic parameter selection

After the runs, 00_convergence/choose_params.py (invoked by `run_convergence.sh`, `analyze_convergence` and the phase-2 scripts) selects the smallest ENCUT and k-mesh from which every subsequent change stays below

$$\max_i |\Delta F_i^{\text{atom } 1}| < 5 \text{ meV/\AA} \quad \text{and} \quad \max_i |\Delta \sigma_{ii}| < 0.5 \text{ kbar},$$

and writes the result to 00_convergence/converged_params.txt. Phase 2 (`run_calculations.sh`, `submit_calculations.sh`, and the GUI form) offers these values as defaults; when run *non-interactively* — e.g. a convergence test automatically followed by the SCF in a chained/nohup run — they are applied without prompting. If the criteria are not met within the tested range,

the largest tested value is used and a NOT CONVERGED warning is issued. Thresholds can be overridden: `choose_params.py -ftol 3 -ptol 0.2`.

13 Parallelization

13.1 Auto-defaults

When KPAR/NCORE are not specified they are set *per step* from two numbers only: the total MPI rank count N and that step's irreducible k-point count n_k (the spglib reduction of the step's own KPOINTS; the LOBSTER NSCF runs ISYM=0, so its full mesh counts).

Quantity	Rule
KPAR	the largest divisor of N that is $\leq n_k$. k-point groups scale almost perfectly, so use as many as there are k-points; a divisor, so no rank is idle.
NCORE	the largest divisor of $r = N/\text{KPAR}$ (ranks per k-group) that is $\leq \sqrt{r}$, VASP's own recommendation (NCORE = 1 for 1–3 ranks per group).

Examples with $N = 40$: $n_k \geq 40 \Rightarrow \text{KPAR} = 40$, $\text{NCORE} = 1$; $n_k = 20 \Rightarrow 20/1$; $n_k = 7 \Rightarrow 5/2$; $n_k = 1 \Rightarrow 1/5$. If only one of the pair is given (instruction key, per-step key or an INCAR block), it is kept — snapped down to a divisor of N — and the other is derived from it: a fixed KPAR gets NCORE from the rule above; a fixed NCORE gets the largest KPAR $\leq n_k$ that leaves a multiple of NCORE ranks per group. Priority, highest first: ELF pin (KPAR = 1) > INCAR block > per-step key (SCF_KPAR, ...) > global KPAR/NCORE > the rule.

Three steps opt out: **DFPT** is always KPAR = NCORE = 1 (linear response supports neither); **phonons** and **Wannier90** default to KPAR = 1 (independent supercell jobs / the Wannier interface), with NCORE from the rule. With 40 ranks and the dense meshes of a small cell (≤ 20 atoms, hundreds of irreducible k-points), KPAR = 40, NCORE = 1 is therefore the expected, correct result; it costs one copy of the wavefunctions per rank, which is small for such cells.

Note. LELF=.TRUE. requires KPAR = 1. With ELF: on (the default) the SCF itself runs LELF and is pinned to KPAR = 1. With ELF: separate the SCF keeps the automatic KPAR and a short KPAR = 1 restart writes the ELFCAR afterwards (Section 11.1). Convergence tests drop LELF automatically.

13.2 Manual overrides

```
MPI: 128          KPAR: 4          NCORE: 8

# Per-step (take priority over globals)
SCF_KPAR: 4       BANDS_KPAR: 1     DOS_KPAR: 8
SCF_NCORE: 8      BANDS_NCORE: 16   DOS_NCORE: 8
RELAX_KPAR: 2     PHONONS_KPAR: 1   # DFPT is always KPAR=1, NCORE=1
```

14 SLURM Cluster Settings

14.1 profiles/slurm.json

```
{
  "vasp_std": "vasp_std",  "vasp_ncl": "vasp_ncl",
  "mpi_cmd":  "srun",
  "modules":  ["intel/2021", "impi/2021", "vasp/6.3.2"],
  "slurm": {
    "partition": "standard",
```

```

    "nodes":          2,
    "ntasks_per_node": 64,
    "time":           "12:00:00",
    "account":         "mygroup"
  }
}

```

14.2 Per-project overrides (in instructions.txt)

```

NODES: 4      NTASKS_PER_NODE: 32      PARTITION: gpu
WALLTIME: 48:00:00      ACCOUNT: myallocation

```

15 Wannier90

```

Tasks: Wannier90 wannierization

WANNIER_NUM_WANN: 8
WANNIER_PROJ: Ga:s;As:p
WANNIER_EWIN: -5.0 to 15.0

```

The agent generates `05_wannier/wannier90.win` with the NSCF k-mesh and projection block. After the NSCF run, execute `wannier90.x` from the `05_wannier/` directory.

16 Born Charges, Dielectric & Piezoelectric Tensors (DFPT)

The DFPT task creates `06_dfpt`, which computes Born effective charges Z^* , the macroscopic dielectric tensor, and the piezoelectric stress tensor from the converged SCF charge density.

Two routes are generated automatically depending on the method:

Method	Route
PBE, PBEsol, LDA, AM05 (no SOC)	Linear response: <code>IBRION=8, LEPSILON=.TRUE.</code>
SOC, HSE06, or R2SCAN	Finite field (PEAD): <code>IBRION=6, LCALCEPS=.TRUE.</code> — VASP's DFPT is not implemented for noncollinear magnetism, hybrids, or meta-GGAs. Requires an insulating system.

Note. Both routes support neither k-point nor band parallelisation: **KPAR** and **NCORE** are forced to 1 in the generated INCAR (overriding any global or per-step setting). The full MPI task count is still used for plane-wave parallelisation. Likewise **ISYM=2** is enforced (symmetry imposed): perturbation theory relies on the symmetry-reduced perturbation set, so a symmetry-off `ISYM=0/-1` — e.g. from a global INCAR: override block intended for the LOBSTER/NSCF steps — is never inherited by `06_dfpt`.

`copy_from_scf.sh` (invoked by `run.sh`) pulls `WAVECAR` and `CHGCAR` from `02_scf` at run time. If `02_scf` has been converted in place for LOBSTER (`ISYM=0/-1` in its INCAR, as done by the high-throughput `run_scf_for_lobster.sh`), the `WAVECAR` copy is skipped — it holds the full, unsymmetrised k-mesh, which the `ISYM=2` run cannot restart from — and VASP regenerates the orbitals from the `CHGCAR`, which the LOBSTER conversion (a fixed-charge NSCF with `LCHARG=.FALSE.`) leaves untouched. If the symmetry-off INCAR is *not* a fixed-charge NSCF, a warning is printed that the `CHGCAR` itself may stem from a symmetry-off SCF and a symmetric SCF should be re-run first.

`run.sh` executes `extract_born.py` after VASP, which writes:

- **BORN** — phonopy NAC format: electronic $\varepsilon_\infty + Z^*$ per atom.
- **born_charges.txt** — human-readable summary shown in the GUI Results tab: electronic (ion-clamped) ε_∞ , ionic contribution, total static $\varepsilon_0 = \varepsilon_\infty + \varepsilon_{\text{ion}}$; Z^* tensors labelled by element; an acoustic-sum-rule check ($\sum_{\text{ions}} Z^*$ should vanish — a large residual indicates an under-converged k-mesh or EDIFF); and the piezoelectric stress tensors e (C/m², clamped-ion, ionic, and relaxed-ion total; meaningful only for non-centrosymmetric crystals).

DFPT_EDIFF in **instructions.txt** tightens the electronic convergence (default 1E-8).

16.1 Example

```
Project: GaAs
Methods: PBE functional
Tasks: relaxation, SCF, DFPT Born charges and dielectric
DFPT_EDIFF: 1E-8
MPI: 16
```

In the GUI, tick **Born Charges & Dielectric Constant (DFPT)** on the Setup tab (the DFPT EDIFF field is optional); after running **06_dfpt** from the Workflow tab, the full summary appears in the **Born charges** panel of the Results tab. With SOC, HSE06 or R2SCAN selected under Methods, the finite-field INCAR is generated automatically — no extra input is needed. For LO-TO splitting in phonon spectra, combine with the phonon task as in the GaAs worked example (Section 19.3).

17 Phonons (phonopy)

```
Tasks: phonons lattice dynamics

PHONONS_DIM: 2 2 2
PHONONS_MESH: 20 20 20
PHONONS_DISP: 0.01
PHONONS_BAND: G M K G A
PHONONS_NAC: FALSE          # disable non-analytic correction
```

Note. For the non-analytic correction (important for polar materials), include DFPT in your task list so Born effective charges are computed.

18 LOBSTER Bonding Analysis (COHP / COBI / COOP)

Adding LOBSTER COHP/COBI analysis (keywords: **lobster**, **cohp**, **cobi**, **coop**, **bonding**) to the task list creates **08_lobster**: a symmetry-off NSCF that reads **02_scf/CHGCAR**, followed by a run of the **lobster** binary.

- **ISYM=0 is required** — a symmetrised k-mesh gives unusable COHP/ICOHP values (wrong sign, errors above 100%). The generated INCAR sets it automatically. With SOC enabled, **ISYM=-1** is used instead: spin-orbit coupling breaks time-reversal symmetry, which ISYM=0 would still assume.
- **NBANDS** is auto-set to at least the LOBSTER basis-function count; **LMAXMIX** is chosen from the elements (6 for *f*, 4 for *d*, 2 for *sp*).

- The LOBSTER binary is set with `LOBSTER_X` in `site.env` (default `lobster-5.1.0-OSX`); it can be overridden at runtime via `$LOBSTER_BIN`.

```
Project: NaCl
Methods: PBE functional
Tasks: SCF, LOBSTER COHP/COBI analysis
MPI: 16
```

Run with `./vasp-agent.py` (builds and runs `02_scf + 08_lobster`); the GUI shows COHP/COBI/COOP plots on the Results tab. To aggregate averaged COHP/COBI over many projects into `lobster_acohp_acobi.csv`, run `python3 lobster_postprocess.py`.

18.1 LOBSTER projected DOS vs. the VASP DOS

The LOBSTER card also carries a projected-DOS plot (`analysis/*_lobster_dos.{png,pdf}`), generated by every interface but displayed only in the GUI), drawn in the same cumulative-stacked layout as the `04_dos` projected DOS. It answers a question the two DOS files raise on their own.

VASP's `LORBIT=11` PDOS projects onto spherical harmonics *inside the PAW augmentation spheres*, which do not fill space, so the interstitial charge is unassigned and $\sum \text{PDOS} < \text{total DOS}$ — severely so for free-electron-like metals (only 25% of the electrons for fcc Al; 72% for MoSe_2 , whose Mo-*d* and Se-*p* states are compact). LOBSTER instead re-expands the PAW states in a local atomic-orbital basis and performs a Mulliken population analysis, so its decomposition is complete by construction: the stacked areas sum *exactly* to LOBSTER's own total.

The plot therefore shows three things: the stacked LOBSTER channels, LOBSTER's total (dashed grey, identically the top of the stack), and the projection-free VASP total from `04_dos` (solid black). The gap between the last two is the charge spilling, reported in `lobsterout` and typically 1–4%. For the *total* DOS the VASP curve is the reference — it is a direct eigenvalue count under the tetrahedron method, with no projection at all — whereas for *orbital-resolved* weights the LOBSTER decomposition is the more complete one, and is the same basis that produces the COHP/COBI curves. Note that the two curves also carry different broadening (`04_dos` uses `ISMear=-5`; the LOBSTER NSCF is forced to Gaussian smearing, and LOBSTER applies its own `gaussianSmearingWidth` on a coarser energy grid), so some of the visible difference is resolution rather than physics. Both smearing settings are stated in the plot legend.

19 Worked Examples

19.1 MoS_2 monolayer with SOC

```
Project: MoS2 monolayer

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Mo-d and S-p)

Convergence: Test k-points 8x8 to 20x20
             ENCUT 400-600 eV

MPI: 64
```

The generator detects hexagonal (hP) and uses G-M-K-G-A-L-H-A.

19.2 Nickel: magnetism + GGA+U

```
Project: Nickel GGA+U

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Ni-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Ni-d and Ni-s)

MAGMOM: 2.0
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 08:00:00
```

With a primitive FCC POSCAR (angles $\approx 60^\circ$) the generator detects cF and uses G-X-W-K-G-L-U-W. Band and DOS plots show both spin channels.

19.3 GaAs: phonons with NAC

```
Project: GaAs phonons

Methods: PBEsol functional

Tasks: SCF calculation
       DFPT Born charges and dielectric
       phonons lattice dynamics

PHONONS_DIM: 4 4 4
PHONONS_MESH: 20 20 20
PHONONS_BAND: G X W L G K

MPI: 64      WALLTIME: 24:00:00
```

19.4 High-pressure relaxation with custom INCAR tags

A constant-pressure cell relaxation at 10 GPa, combined with raw INCAR passthrough to tighten convergence and add tags the parser has no dedicated keyword for:

```
Project: Silicon 10GPa

Methods: PBEsol functional

Tasks: structure relaxation
       SCF calculation
       band structure

PRESSURE: 10 GPa          # constant-P relax -> ISIF=3, IBRION=2,
PSTRESS=100
```

```
# Tags applied to every step
INCAR:
  LREAL = .FALSE.
  ADDGRID = .TRUE.
  LMAXMIX = 4
END_INCAR

# Tighter electronic convergence for the relaxation only
INCAR relax:
  EDIFF = 1E-8
  EDIFFG = -0.005
END_INCAR

ENCUT: 600
MPI: 64
```

The 01_relax/INCAR is generated with ISIF=3, PSTRESS=100, the global tags appended, and EDIFF/EDIFFG overridden to the per-step values; 02_scf and 03_bands receive only the global block.

19.5 Bi₂Se₃: topological insulator

```
Project: Bi2Se3 topological

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Bi-p and Se-p)
       Wannier90 wannierization

Convergence: Test k-points from 8x8x4 to 16x16x8
             ENCUT 400-600 eV

WANNIER_NUM_WANN: 18
WANNIER_PROJ: Bi:p;Se:p
WANNIER_EWIN: -2.0 to 2.0
MPI: 128
```

20 Common OUTCAR Checks

grep "reached_required_accuracy"	OUTCAR	# converged?
grep "energy_without"	OUTCAR tail -1	# total energy
grep "E-fermi"	OUTCAR tail -1	# Fermi level
grep "magnetization(x)"	OUTCAR tail -5	# moment
grep "TOTAL-FORCE"	OUTCAR tail -20	
grep -A3 "BORN_EFFECTIVE_CHARGES"	OUTCAR head -20	

21 Troubleshooting

Symptom	Solution
POTCAR not found	Check <code>VASP_POTCAR_DIR</code> in <code>site.env</code> or GUI System Setup. Use the POTCAR chooser for unusual variants (<code>Ga_d</code> , <code>Fe_sv</code>).
<code>vasp_std</code> not found	Set the full binary path in <code>site.env</code> or GUI System Setup.
WARNING: may not have converged	Increase NSW (ionic) or NELM (electronic) in INCAR and re-run.
Wrong bands ($W = \Gamma$)	Conventional cubic cell detected; use the <i>primitive</i> cell for FCC/BCC so auto-detection picks cF/cI.
Projected DOS missing	Check <code>instructions.txt</code> : <code>projected on</code> and <code>projections on</code> are both accepted. Verify element names match POSCAR.
KPAR error	$KPAR \times NCORE$ must divide total MPI tasks. Use auto-defaults or set explicitly valid values.
GUI unreachable	Check SSH tunnel: <code>ssh -L 5001:localhost:5001 user@cluster</code> .
Project missing from dropdown	Folder must contain POSCAR + <code>settings.json</code> in the directory where <code>vasp-gui.py</code> was started.
Band plot empty or not shown	Usually <code>sumo-bandplot</code> crashed on this band structure; the GUI now falls back to a built-in plotter automatically. An empty plot also means a wrong E_F : the Fermi level is read from <code>04_dos/OUTCAR</code> (then <code>02_scf/OUTCAR</code>), since <code>03_bands</code> eigenvalues share the NSCF/DOS energy reference.

22 Version History

- v1.0 (2026-03-23)** Initial release — GUI, workstation agent, PBEsol/R2SCAN/HSE06, SOC, GGA+U, convergence tests, Wannier90, phonopy.
- v1.1 (2026-04-05)** Per-task KPAR/NCORE smart defaults; SLURM agent with dependency-chained submission; cumulative DOS by element (no pymatgen); copy-from-relax/scf scripts.
- v1.2 (2026-06)** Collinear magnetism: MAGMOM per atom, spin-resolved band and DOS plots. Auto k-path detection (8 crystal systems). Fixed DOS projection parsing. Separate GUI and agent quick-reference sheets. Rewritten comprehensive LaTeX manual.
- v1.3 (2026-06-15)** Raw INCAR passthrough (`INCAR: ... END_INCAR`, global or per-step) for arbitrary VASP tags. Constant-pressure relaxation via `PRESSURE` (`ISIF=3`, `IBRION=2`, `PSTRESS`).
- v1.4 (2026-06-19)** High-throughput driver `ht-mp-scf.py`: Materials Project primitive cells → batch SCF + ELF, with per-material element/*Z* table and cross-material `afterok` chaining on SLURM.
- v1.5 (2026-06-24)** Robust band plotting across the GUI and both CLI agents (workstation + SLURM): the built-in `vasprun.xml` plotter now handles non-spin as well as spin, and is used automatically when `sumo-bandplot` crashes or is unavailable. Fixed the matplotlib fallback (`plot_results.py`) to resolve the real Fermi level and emit PNG, not just PDF.

- v1.6 (2026-07-08)** Band k-path derived from the real space group via spglib (Setyawan–Curtarolo) in all agents and the GUI, with the geometric classifier kept as fallback. New `08_lobster` step: LOBSTER COHP/COBI/COOP bonding analysis with GUI plots and the `lobster_postprocess.py` aggregator; the NSCF uses `ISYM=0`, or `ISYM=-1` when SOC is on. Full GPL-3.0 text in `LICENSE` (GitHub now detects the license correctly). Manual and quick-reference sheets brought up to date.
- v1.7 (2026-07-08)** DFPT step overhaul: KPAR/NCORE forced to 1 (linear response and PEAD support neither); automatic finite-field route (`LCALCEPS`, `IBRION=6`) when SOC, HSE06 or R2SCAN is requested; `extract_born.py` now reports ϵ_∞ , the ionic contribution, total static ϵ_0 , an acoustic-sum-rule check, element-labelled Z^* , and clamped-/relaxed-ion piezoelectric tensors.
- v1.8 (2026-07-09)** ELF bond profiles: `elf_bonds.py` plots the electron localization function along non-equivalent nearest-neighbour bonds (atom centre to atom centre), wired into `analyze.sh` for both CLI agents and shown in the GUI Results tab whenever `02_scf/ELFCAR` exists. Output is named after the reduced formula (`MoSe2_elf_bonds.pdf`). Detects the all-zero ELFCAR that VASP writes for non-collinear / spin-orbit runs and reports it instead of emitting a blank plot.
- v1.9 (2026-07-22)** *K-mesh and GGA+U*. Physics-based cell-aware k-mesh generation replacing fixed uniform grids. Two density tiers: **coarse** (1000 k-points per reciprocal atom, kpra) for convergence tests and relaxations; **fine** (5000 kpra) for SCF, DOS, and spectroscopic steps. Subdivisions $N_i \propto |\mathbf{b}_i^*|$ from reciprocal lattice vectors, all-even to include BZ boundary points; $N_z = 1$ for 2-D slabs. Hexagonal systems: fixed 6×6 (coarse) or 12×12 (fine) in-plane, N_z from $|\mathbf{b}_3^*|/|\mathbf{b}_1^*|$. DOS mesh always $2N_x \times 2N_y \times 2N_z$. DFPT forced to match SCF mesh exactly. New instruction key `KMESH_DENSITY` (coarse/fine). GGA+U anion gate extended to all chalcogenides and halides (O, F, S, Se, Te, Cl, Br, I); Hubbard lookup table now covers sulfides such as AgCrP_2S_6 .
- Plot improvements.* ELF bond-profile and LOBSTER (COHP/COBI/COOP) plot legends and statistics boxes moved outside the axes area so they never obscure curves. Shared plotting module `modules/cohp_plot.py` used by both the GUI and the CLI agents; COHP, COBI, and COOP plots are now generated automatically in `analyze.sh` for both the workstation and SLURM agents, matching GUI behaviour.
- Structure visualizer.* New *Visualize* button in the POSCAR card: converts the current POSCAR to XYZ via `modules/poscar2xyz.py` and opens it in Jmol, VESTA, or any other viewer. The viewer path is set under System Setup $\rightarrow$ *Structure visualizer* (or `VISUALIZER_X` in `default_paths`).
- Configuration.* `LOBSTER_X` and `VISUALIZER_X` are now exposed in the GUI System Setup page and propagated to the agent subprocess environment. Platform config file renamed from `site.env` to `default_paths` for clarity; three-layer precedence (`default_paths < GUI settings < profile JSON`) documented. Auxiliary Python scripts (`band_plot.py`, `dos_plot.py`, `elf_bonds.py`, `lobster_postprocess.py`, `conv_choose.py`, `poscar2xyz.py`) consolidated into `modules/`.
- v2.0 (2026-09-28)** *Input-consistency release*. **MAGMOM**: now honours negative values, per-element ($\text{Fe}=4.0$, $0=0.6$) and per-atom lists, switches `ISPIN` on by itself, and initialises SOC runs along the requested x/y/z direction. K-mesh density is reflected in every KPOINTS file: coarse/fine/integer kpra, even $N_i \propto |\mathbf{b}_i^*|$ chosen for uniform spacing and closeness to the target, hexagonal in-plane meshes in multiples of 6, `RELAX_KMESH_DENSITY`; one

generator routine (`kpoints_text`) serves both agents and the GUI. KPAR/NCORE: ELF-pinned KPAR=1 re-derives NCORE, and values given in INCAR blocks steer the companion value. Relaxation starts from scratch and writes WAVECAR/CHGCAR; the SCF reads CHGCAR, and WAVECAR too when the k-mesh matches. Explicit INCAR `<step>`: and new KPOINTS `<step>`: blocks override everything (GUI: Setup → Advanced). New `check_agent_consistency.py` compares GUI, workstation and SLURM outputs. New `ht-semimetals.py` builds the `highthroughput_*` SCF/bands/LOBSTER screening sets from list files.

v2.1 (2026-09-30) *Simpler k-mesh and parallel rules; two-step ELF.* k-mesh: one formula, N_i = nearest even (hexagonal in-plane: multiple of 6) of $s|\mathbf{b}_i^*|$, replacing the tolerance search. KPAR = largest divisor of the ranks $\leq n_k$; NCORE = largest divisor of the ranks per k-group $\leq$ its square root. Wannier90 and phonons now really default to KPAR = 1, and LOBSTER uses its own (ISYM=0) k-count. New ELF: **separate**: SCF at full KPAR, then a short KPAR = 1 restart writes ELFCAR (default in `ht-mp-scf.py`, which also gains `--kptra` (default coarse), `--kpar`, `--ncore`). `ht-semimetals.py`: `--kpar/--ncore`, POTCARs built automatically, DFPT step removed; each job runs `analyze.sh` at the end (ANALYSIS_PYTHON in `env.sh`), plus `analyze_all.sh`.

Addendum: High-Throughput Quick Reference

Cheat-sheet for `ht-mp-scf.py` (full description in Section 5). For each Materials Project ID it downloads the **primitive** cell, stages an SCF calculation (automatic KPAR/NCORE) followed by a short KPAR = 1 ELF restart (→ ELFCAR) on a uniform coarse Γ -mesh (1000 kptra default; override with `--kptra`, `KMESH_DENSITY` or `--kmesh`), and builds a `runall.sh`.

```

pip install pymatgen mp-api
export MP_API_KEY=...           # Materials Project key
export VASP_POTCAR_DIR=...      # POTCAR library (used by the agent)

printf 'mp-149\nmp-2534\n' > highthrouput_list # one mp-id per line

./ht-mp-scf.py --agent local    --mpi 16        # workstation,
    sequential
./ht-mp-scf.py --agent slurm    --profile slurm  # cluster,
    afterok-chained
./ht-mp-scf.py --agent slurm    --no-chain       # cluster,
    independent
./ht-mp-scf.py --agent slurm    --kptra 8000 --nodes 1
    --ntasks-per-node 40

./runall.sh                     # build inputs + run/submit each SCF

```

Output	Location
ELFCAR (electron localization)	<mp-id>/02_scf/ELFCAR (ELF pass in 02_scf/elf/)
Staged inputs (POSCAR + instructions)	_ht_inputs/<mp-id>/
Element / Z table	_ht_inputs/elements_Z.txt

Note. Transfer to a workstation: only the project folder is needed (code, modules/, profiles/); VASP binaries and the POTCAR library already live there. `ht-mp-scf.py` needs internet for the MP download, so run it on the login node — `runall.sh` does none.

Note. If the download step errors with 'dict' object has no attribute 'number': that is an spglib / pymatgen version mismatch. Fix the root cause with `pip install -U spglib`. The driver also falls back to `Structure.get_primitive_structure()` automatically, so a run is never blocked by it.